



Παράλληλος και Κατανεμημένος Υπολογισμός I

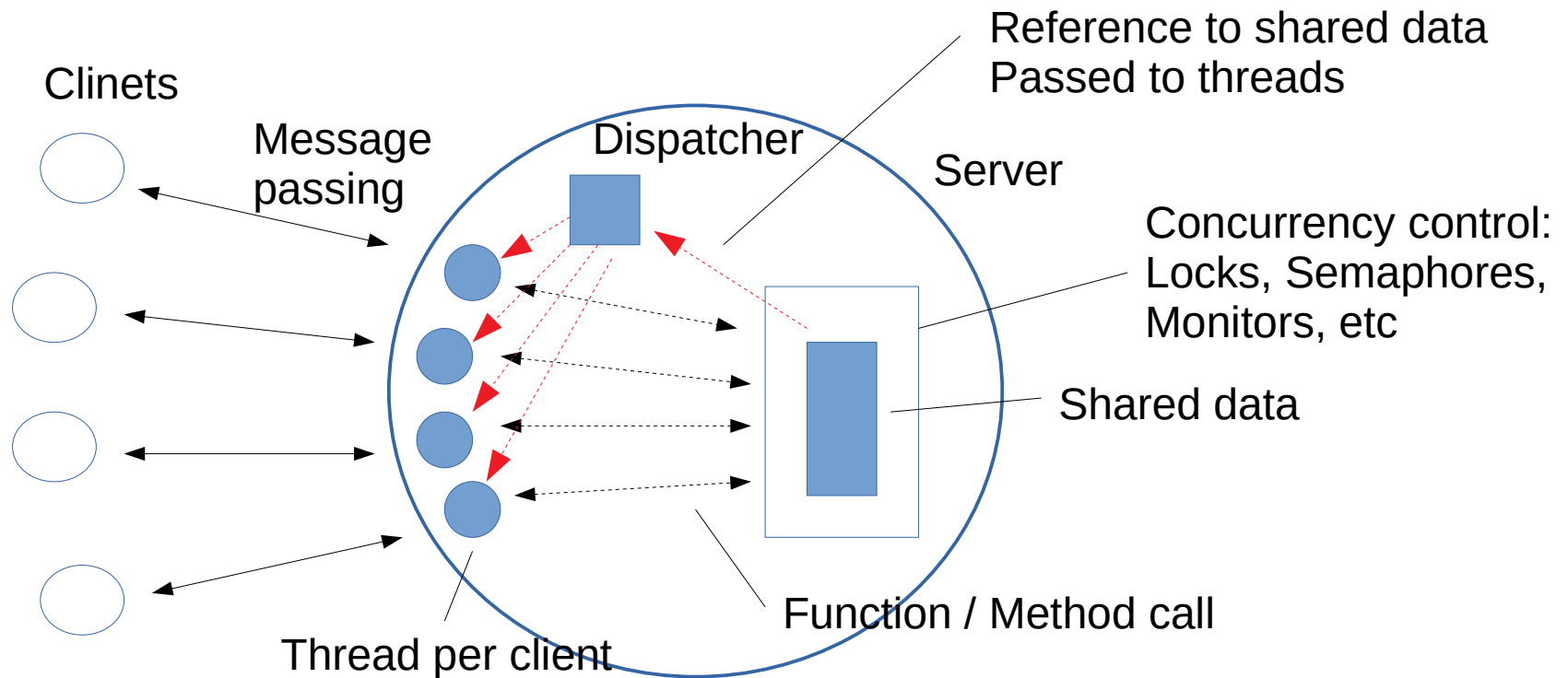
Κατανεμημένη Μνήμη Μεταβίβαση Μηνυμάτων (συν. 2)

Κ.Γ. Μαργαρίτης
Πανεπιστήμιο Μακεδονίας
Τμήμα Εφαρμοσμένης Πληροφορικής
2024



Μοιραζόμενη κατάσταση

Μοιραζόμενη κατάσταση (shared state) 1



- Ένας πολυνηματικός διακομιστής μπορεί να διαθέτει δεδομένα που μοιράζονται τα νήματα που εξυπηρετούν συνδέσεις. Μοιραζόμενη κατάσταση διακομιστή.
- Τα μοιραζόμενα δεδομένα πρέπει να είναι γνωστά στα νήματα και να προστατεύονται με μηχανισμούς ταυτοχρονισμού.

```
//shared variable definition
struct/class shared {
    lock_type lock;
    int data;
}
```

Μοιραζόμενη κατάσταση 2

```
// main
MultiThreadedServerTCPwithSharedState
{
    my_struct = new shared (my_lock, my_data);
    Socket(Port); Bind(Socket); Listen(Socket);
    while (true) {

        connection = Accept (Socket);
        //accept returns an ephemeral connection Socket so
        //Server Socket remains free
        //connection establishes in, out streams

        dispatch serverThread (connection, my_struct);
        //dispatch = create and run a server
    }
```

Μοιραζόμενη κατάσταση 3

```
ServerSideProtocol (shared my_struct)
{
    ... concurrent access to my_struct
}

Server (my_connection, shared my_struct)
{
    //my_connection establishes in, out streams

    while (true) {
        Receive/Read (messageFromClient, in);
        messageToClient =
            ProcessRequestandPrepareReply
                (messageFromClient, my_struct);
        if (messageToClient == exitMessage) break;
        Send/Write (messageToClient, out);
    }
    Close Socket();
}
}
```

Μοιραζόμενη κατάσταση 4

- Οι τεχνικές διαμοιρασμού είναι οι γνωστές: καθολικές μεταβλητές, όρισμα αναφοράς ή αντικειμενοστρεφής (ή και χρήση static μεταβλητών ή μεθόδων).
- Μοιραζόμενη κατάσταση μπορεί να έχει και ένας πολυνηματικός πελάτης. Και πάλι οι τεχνικές διαμοιρασμού και η δομή του κώδικα είναι αντίστοιχα.

Very Simple Chat: Server 1

- Ξεκινούμε από το Multithreaded Echo Server
- Ο διακομιστής αναμένει δύο πελάτες να συνδεθούν και δημιουργεί τα αντίστοιχα νήματα όπου περνά και τις δύο υποδοχές.
- Στη συνέχεια περιμένει για το επόμενο ζεύγος πελατών.
- Μπορεί να χρησιμοποιηθεί σαν βάση για παιχνίδια δύο παικτών (πχ τρίλιζα, ντάμα, σκάκι, ναυμαχία)

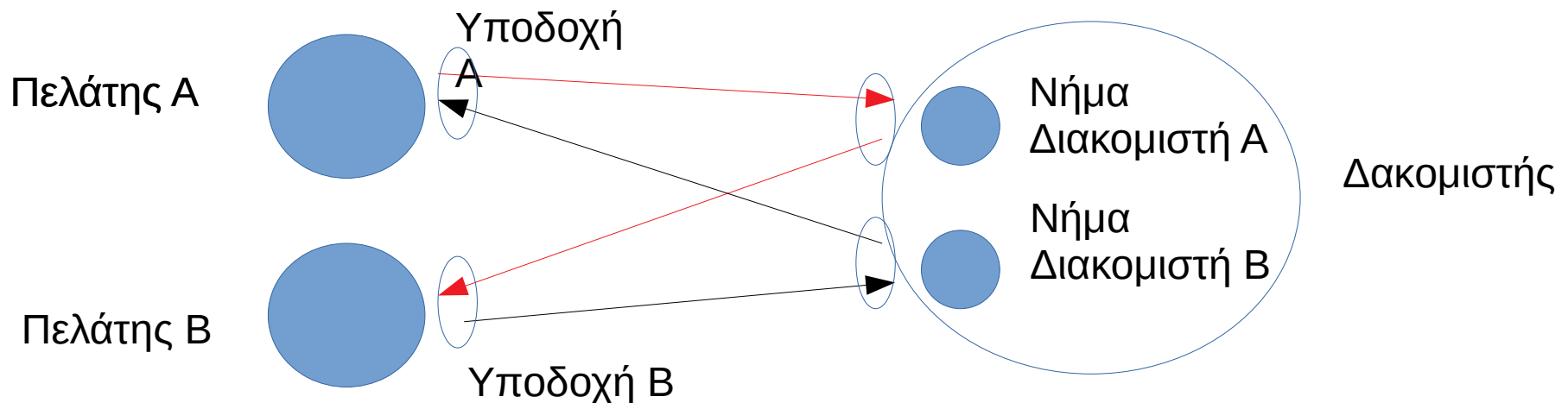
Very Simple Chat: Server 2

ChatServer

```
{  
    Socket(Port); Bind(Socket); Listen(Socket);  
    while (true) {  
  
        connection1 = Accept (Socket);  
        connection2 = Accept (Socket);  
  
        dispatch serverThread (connection1, connection2);  
        dispatch serverThread (connection2, connection2);  
    }  
}
```

Very Simple Chat: Server Thread

- Ξεκινούμε από το Multithreaded Echo Server Thread
- Τόσο η δομή του κώδικα όσο και το πρωτόκολλο είναι ίδιο με το Echo.
- Η μόνη διαφορά: τα ρεύματα επικοινωνίας
- Είσοδος από την υποδοχή του ενός πελάτη και έξοδος από την υποδοχή του άλλου πελάτη.



Very Simple Chat: Client 1

- Ξεκινούμε από το Echo Client
- Ο βρόχος του πελάτη υποθέτει αυστηρή εναλλαγή ανάγνωσης – εγγραφής – ανάγνωσης – εγγραφής -..
- Αυτό είναι επιθυμητό σε παιγνίδια (σειρά σου-σειρά μου) αλλά δεν είναι επιθυμητό στο Chat, γιατί ο ένας χρήστης μπορεί να είναι πολύ πιο ομιλητικός από τον άλλο.
- Λύση: δύο νήματα, ένα για ανάγνωση και άλλο για εγγραφή. Οι δύο λειτουργίες εκτελούνται ταυτόχρονα.

Very Simple Chat: Client 2

```
ChatClient {
    Socket();
    connection = Connect (Server);
    //connection between two sockets, Server address known
    //connection establishes in, out streams

    dispatch readThread (in);
    dispatch writeThread (out);
}
ReadThread (in) {
    while (true) {
        Read/Receive (messageFromServer, in);
        processReply(messageFromServer);
    }
}
WriteThread (out) {
    while (true) {
        messageToServer = prepareRequest();
        Write/Send (messageToServer, out);
        if (message == exitMessage)...//Client Closes Socket;
    }
}
```

Επεκτάσεις: Chat

- Επέκταση σε πολλαπλούς συνομιλητές: αλλαγές στο διακομιστή
 - (α) μοιραζόμενη δομή αποθηκεύει τις υποδοχές
 - (β) κάθε νήμα διαβάζει από ένα πελάτη και γράφει σε όλους τους άλλους.
- Μπορούμε εκτός από υποδοχές να κρατάμε και άλλα στοιχεία (πχ συσχέτιση IP με ψευδώνυμο)
- ...Πολλά άλλα

Επεκτάσεις: Games

- Στα παιχνίδια ανά ζεύγη το μόνο που αλλάζει είναι ο πελάτης. Είναι όμοιος με τον Echo Client μόνο που ένας πελάτης πρέπει να ξεκινά το παιχνίδι ενώ ο άλλος να παίζει δεύτερος.
- Ο διακομιστής κρατά την κατάσταση του παιχνιδιού και ελέγχει κανόνες, τερματισμό κλπ. Δέχεται από έναν παίκτη μια κίνηση, ενημερώνει την κατάσταση και στέλνει την νέα κατάσταση στον άλλον παίκτη.
- Στα multiplayer games η συμπεριφορά του πελάτη εξαρτάται από τους κανόνες του παιχνιδιού.
- Σε multiplayer games η μοιραζόμενη κατάσταση του διακομιστή πρέπει να προστατεύεται γιατί πολλοί παίκτες μπορούν να παίξουν ταυτόχρονα.



Αρχιτεκτονική 3 επιπέδων

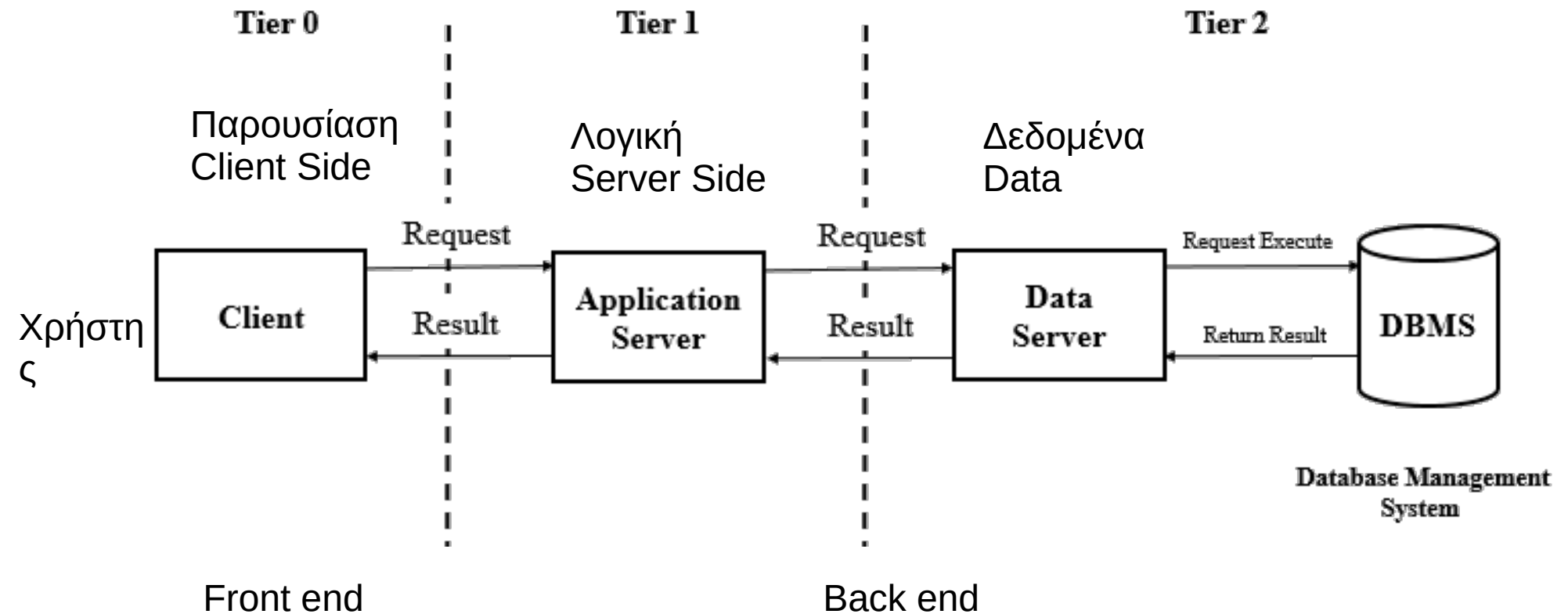
Αρχιτεκτονική 3 επιπέδων 1

- Στη συντριπτική πλειονότητα τα σύγχρονα καταναεμημένα συστήματα είναι **πληροφοριακά συστήματα βασισμένα στον Ιστό** (Web-based information systems), οργανωμένα σε αρχιτεκτονικές 3-Επιπέδων (3-Tier):
- Επίπεδο **Παρουσίασης (Presentation)**: συνήθως έχει τη μορφή εφαρμογής που εκτελείται ως πελάτης (client-side εφαρμογή) σε ένα περιηγητή ιστού (ή web app ή mobile app)

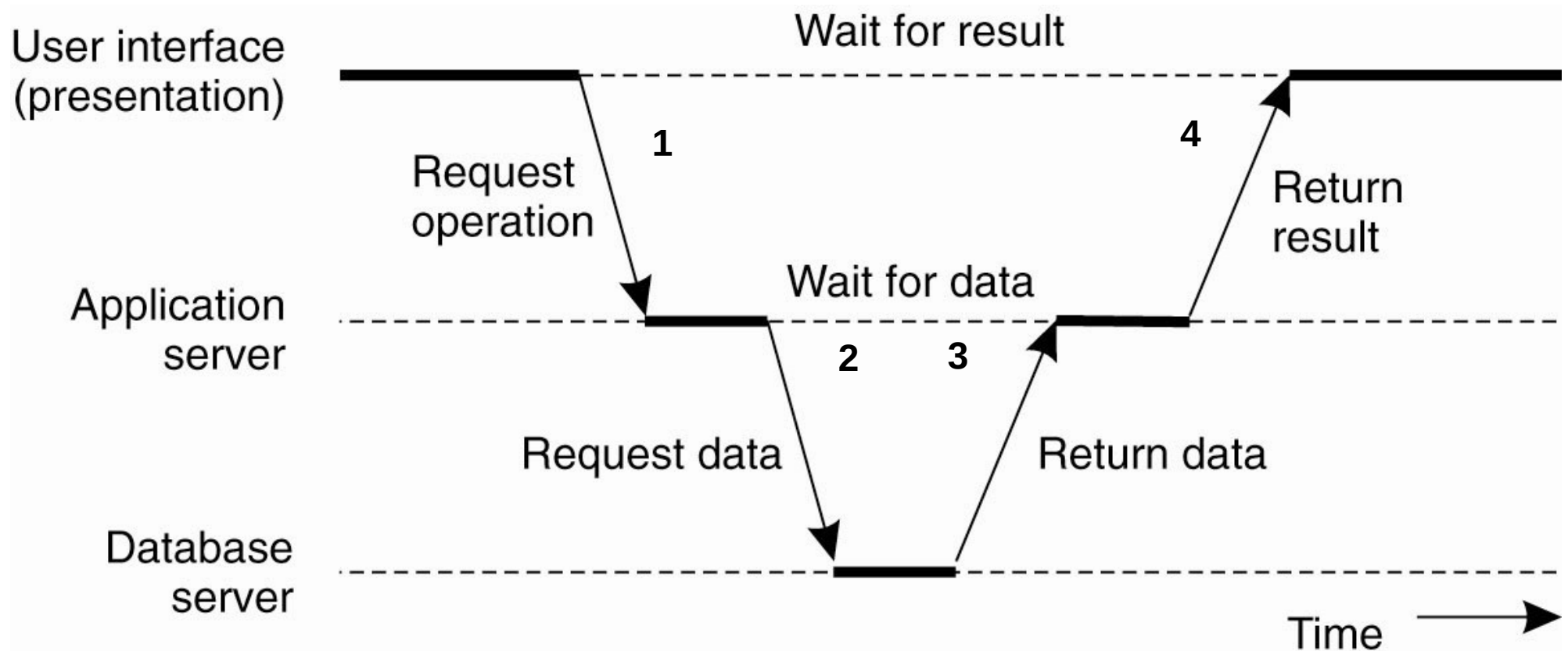
Αρχιτεκτονική 3 επιπέδων 2

- Επίπεδο **Λογικής (Logic)**: συνήθως έχει τη μορφή εφαρμογής που εκτελείται σε ένα διακομιστή ιστού (server-side εφαρμογή) με χρήση PHP, JSP, ASP κλπ και σύνδεση με βάση δεδομένων
- Επίπεδο **Δεδομένων (Data)**: συνήθως ένας διακομιστής βάσεων δεδομένων που επικοινωνεί με τη server-side εφαρμογή (λέγεται και back-end).

Αρχιτεκτονική τριών επιπέδων 3



Αλληλεπίδραση πελάτη διακομιστή σε αρχιτεκτονική 3 επιπέδων



Αρχιτεκτονική 3 επιπέδων: διακομιστής εφαρμογής

```
ServerSideProtocol {...}
Server
{
    ServerDataBaseConnection = ConnectToDataBase(DataBase)
    while (true) {

        clientServerConnection = accept (someClient);
        while (true) {
            ...
            fromClient = read (clientServerConnection);    //1
            toDataBase = processClientRequest (fromClient);
            write(toDataBase, serverDataBaseConnection);    //2
            fromDataBase = read (serverDataBaseConnection); //3
            toClient = replyToClient(fromDataBase);
            write (messageToClient, clientServerConnection); //4
            ...
        }
    }
}
```

Ο application server έχει διπλό ρόλο: διακομιστής αλλά και πελάτης

Βασικές λειτουργίες αρχιτεκτονικής 3 επιπέδων

- Create, Read, Update, Delete
Δημιουργία, Ανάγνωση, Ενημέρωση, Διαγραφή

Message

ή HTTP

Create
Read (Retrieve)
Update (Modify)
Delete (Destroy)

PUT / POST
GET
PUT / POST
DELETE

SQL

INSERT
SELECT
UPDATE
DELETE

Πρωτόκολλο εφαρμογής πελάτη και
διακομιστή εφαρμογής

Πρωτόκολλο εφαρμογής
διακομιστή εφαρμογής και
διακομιστή βάσης δεδομένων